
Trap Handling in xv6

xv6 trap mechanism

Contents

- Trap overview
- Key Components of Trap Handling
- Handling Sequence for Traps from User Space
- Handling Sequence for Traps from Kernel Space

Trap Overview

What is trap?

Trap in xv6

- **Trap in xv6:**

- A **pause** in normal program execution **to handle special events**.
- It's slightly different from what you learned in the theory class. In xv6, 'trap' is an overarching concept that **encompasses** traps, interrupts, and exceptions.

- **Three Main Causes of Traps**

- **System call:**

- Functions invoked when the user program wants to use kernel services.
 - e.g., `read()`, `write()`, `fork()`, `ps()` ...

- **Exception**

- Illegal or unexpected action generated within a user program.
 - e.g., divide by zero, use an invalid virtual address, ...

- **Interrupt**

- Asynchronous notification from a hardware device.
 - e.g., disk hardware finishes a read or write request, timer interrupt

What the OS Must Do During a Trap

- The operating system must support following tasks for trap handling
 1. Saving current processor's registers
 2. Setting up for kernel execution
 3. Retrieving information about the event
 4. Maintaining isolation between user and kernel
- To meet these requirements, xv6 uses different trap handlers depending on whether the trap occurred in user mode or kernel mode.
 - user mode : `uservec` in `kernel/trapline.S`
 - kernel mode : `kernelvec` in `kernel/kernelvec.S`

Key Components of Trap Handling

Special registers and data structures

Three RISC-V privilege modes

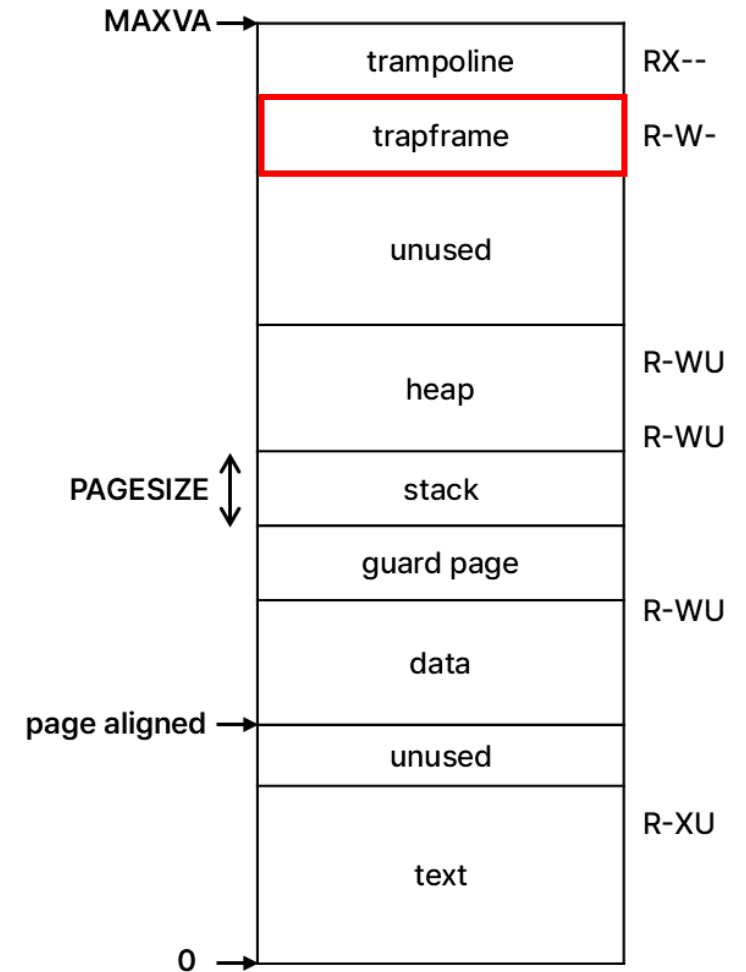
- Machine Mode
 - CPU starts in machine mode
- Supervisor Mode
 - Allowed to execute privileged instructions
 - The operating system kernel runs in supervisor mode
- User Mode
 - User processes run in user mode

Control and Status Register (CSR)

- Special purpose registers used to configure processor behavior and monitor system status.
- **stvec**
 - Holds trap handler address.
 - Directs the CPU to jump to this address when a trap occurs.
- **sepc**
 - Saves program counter at the time of the trap.
 - The CPU can resume execution after trap handling using this address.
- **scause**
 - Stores trap reason code.
 - Identifies exactly what caused the trap.
- **sstatus**
 - Remembers the CPU's previous state and manages interrupt handling setting using three bits.
 - **SPP**: Records whether the CPU's previous state is user or supervisor.
 - **SIE**: Determines whether the kernel accepts or blocks external interrupts now.
 - **SPIE**: Backs up the SIE state exactly as it was just before the trap occurred.
- **sscratch**
 - Temporary storage for the trap handler.
 - Helps safely save user registers in TRAPFRAME.

What is a TRAPFRAME

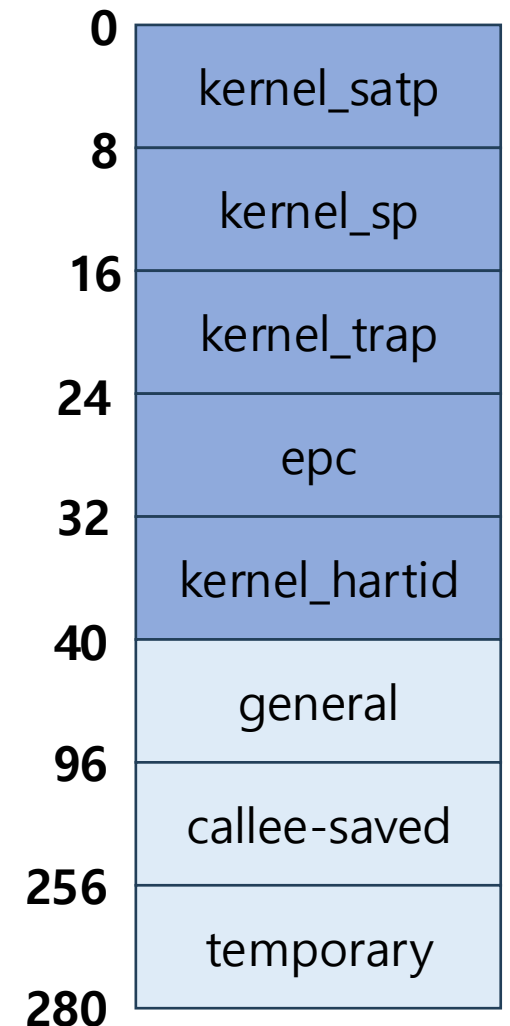
- A predefined memory area designated to store data necessary for trap handling.
 - Holds relevant **kernel data** needed during trap transition.
 - Acts as a backup area for **user registers** (CPU state) during trap handling.
- Exactly **one page (4096 bytes)** of space is allocated **for every process**.
- Defined as `struct trapframe` in the `kernel/proc.h` for easy access.



A process's user address space

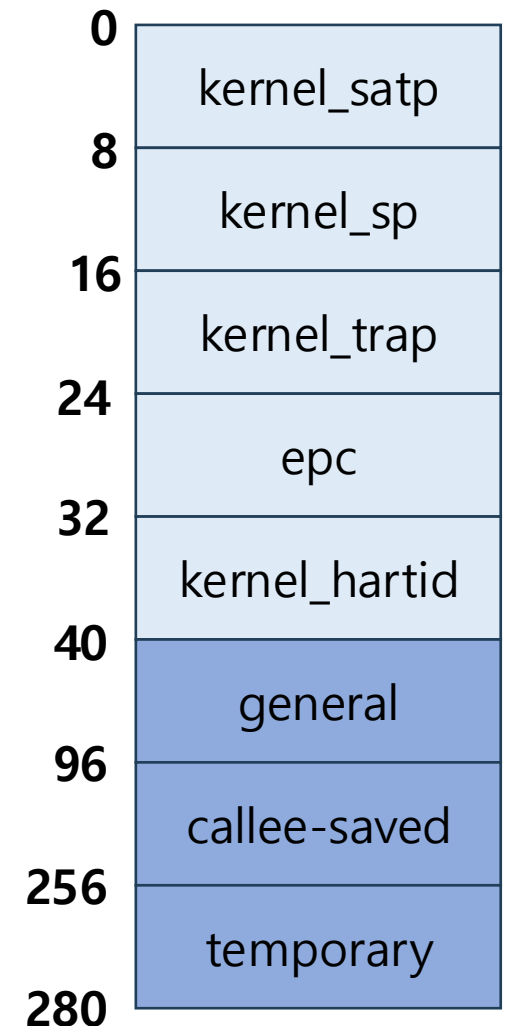
Essential Data for Trap Transition in TRAPFRAME

- **kernel_satp**: Stores the address of the kernel page table.
- **kernel_sp**: Stores top address of the process's kernel stack.
- **kernel_trap**: Stores the address of the `usertrap()` function.
- **epc**: Stores program counter(pc) of the process to return to later.
- **kernel_hartid**: Stores ID of the CPU core (hart ID) executing the process.



Backup area for user registers in TRAPFRAME

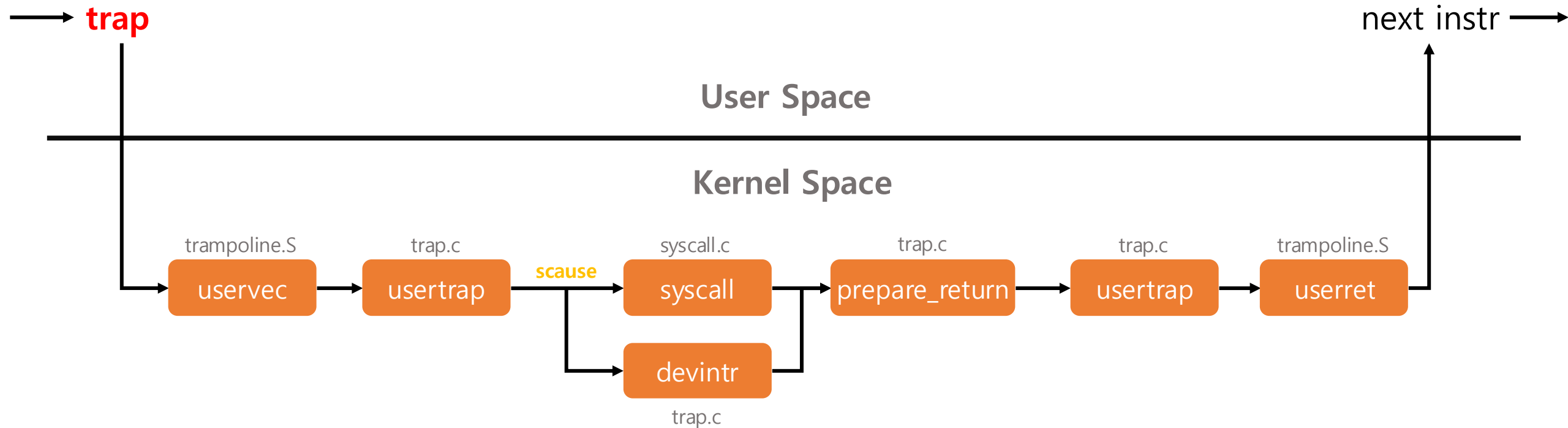
- **general registers:** Stores the values of the registers currently in use.
 - contains ra, sp, gp, tp, t0~t2
- **callee-saved registers:** Stores the registers used by the called function.
 - contains s0~s11, a0~a7
- **temporary registers:** Stores the temporary register value used when traps occurred.
 - contains t3~t6



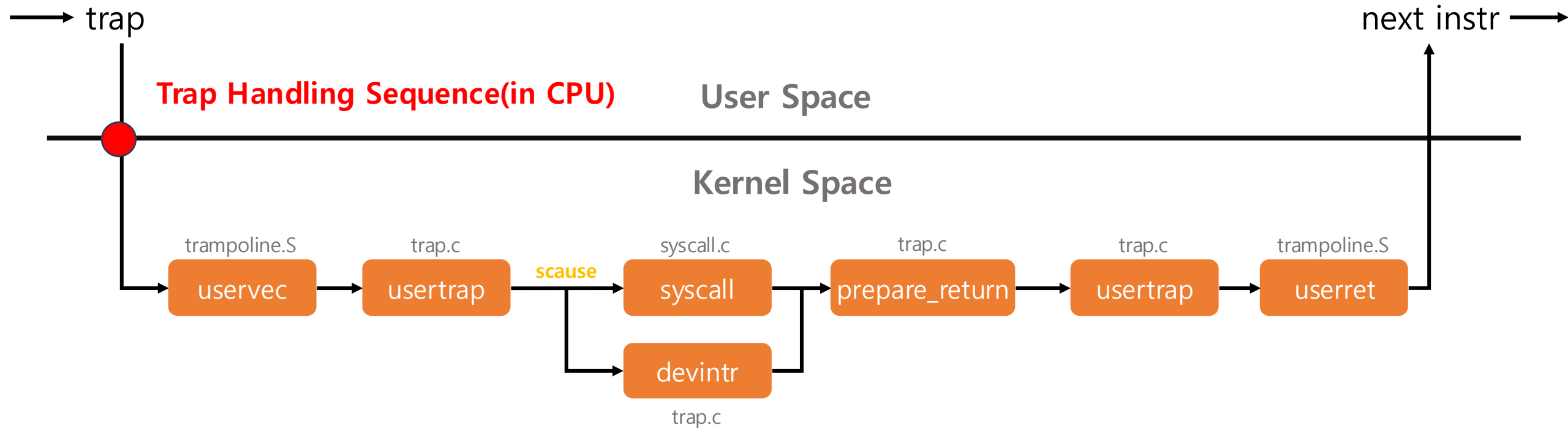
Traps from User Space

user space trap flow

Traps from User Space



Traps from User Space



Trap Handling Sequence

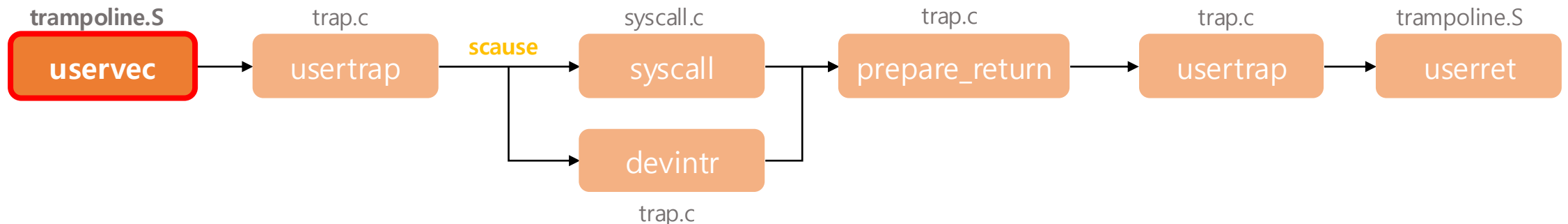
If the trap is a device interrupt, and the **sstatus** SIE bit is clear, don't do any of the following.

1. Disable interrupts by clearing the SIE bit in **sstatus**.
2. Copy the program counter to **sepc**.
3. Save the current mode (**user** or supervisor) in the SPP bit in **sstatus**.
4. Set **scause** to reflect the trap's cause.
5. Set the mode to supervisor.
6. Copy **stvec** to the program counter.
7. Start executing at the new program counter.

uservec

1. Save the **user's a0** register to **sscratch**.
2. Load the **TRAPFRAME** address into **a0**.
3. Save the remaining user registers to **TRAPFRAME**.

```
$ vim kernel/trampoline.S  
  
22  uservec:  
...  
32      csrw sscratch, a0  
...  
37      li a0, TRAPFRAME  
...  
40      sd ra, 40(a0)  
...  
69      sd t6, 280(a0)  
...  
72      csrr t0, sscratch  
73      sd t0, 112(a0)
```



uservec

4. Load kernel execution data from **TRAPFRAME**.

1. Initialize kernel stack pointer (`kernel_sp`) *line 76*
2. Make `tp` to hold the current hartid (`kernel_hartid`) *line 79*
3. Load the address of `usertrap()` (`kernel_trap`) *line 82*
4. Fetch the kernel page table address (`kernel_satp`) *line 85*

```
$ vim kernel/trampoline.S
```

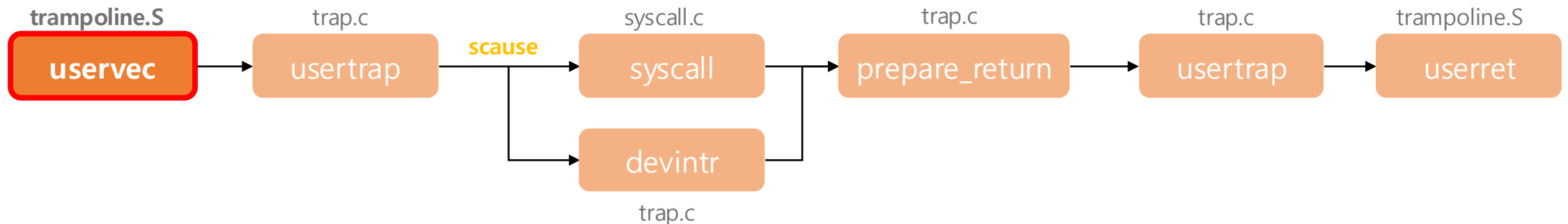
```
22  uservec:
```

```
...  
76      ld sp, 8(a0)
```

```
...  
79      ld tp, 32(a0)
```

```
...  
82      ld t0, 16(a0)
```

```
...  
85      ld t1, 0(a0)
```

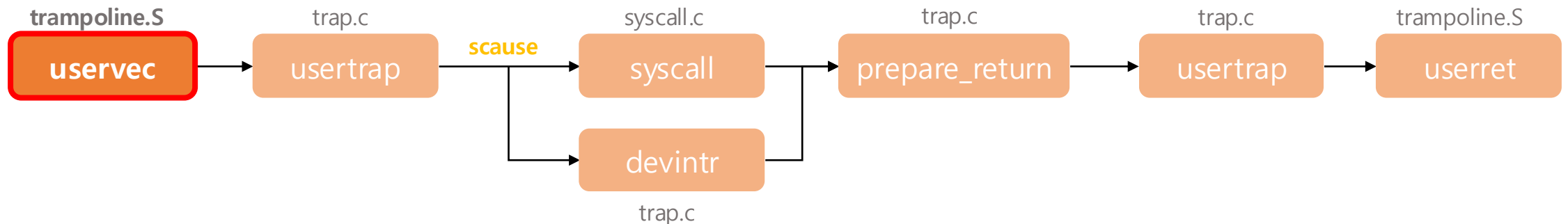


uservec

5. Switch to the **kernel page table**.
6. Jump to `usertrap()`.

```
$ vim kernel/trampoline.S

22  uservec:
...
91  # install the kernel page table
92  csrw satp, t1
...
97  # jump to usertrap()
98  jr t0
```

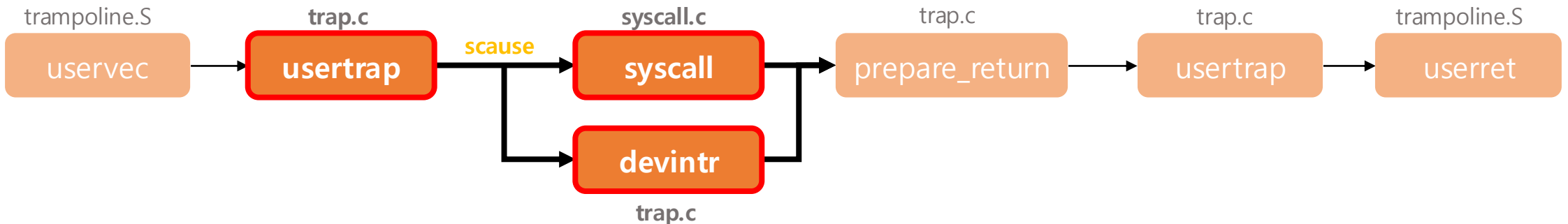


usertrap

1. Change **stvec** to point to `kernelvec`.
2. Save user program counter from **sepc** into **TRAPFRAME**.

```
$ vim kernel/trap.c

38 usertrap(void)
...
44 //send interrupts and exceptions to ...
45 //since we're now in the kernel.
46 w_stvec((uint64)kernelvec);
...
50 //save user program counter.
51 p->trapframe->epc = r_sepc();
```



usertrap

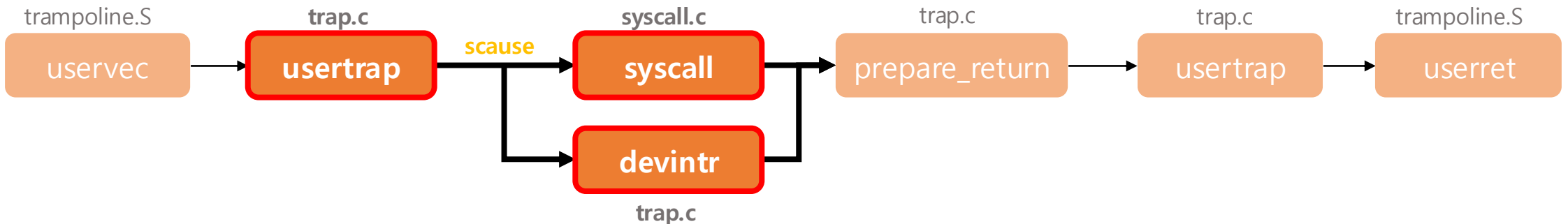
3. Handle different trap types based on **scause**.

- **System calls** → `syscall()`
- **Device Interrupt** → `devintr()`
- **Exceptions** → process termination

4. Call `prepare_return()`.

```
$ vim kernel/trap.c
```

```
54  if(r_scause() == 8){  
...  
68      syscall();  
69  } else if((which_dev = devintr()) != 0){  
...  
74  } else {  
75      printf("usertrap(): unexpected scause...  
76      printf("                sepc=0x%lx stval=...  
77      setkilled(p);  
78  }  
...  
87  prepare_return();
```

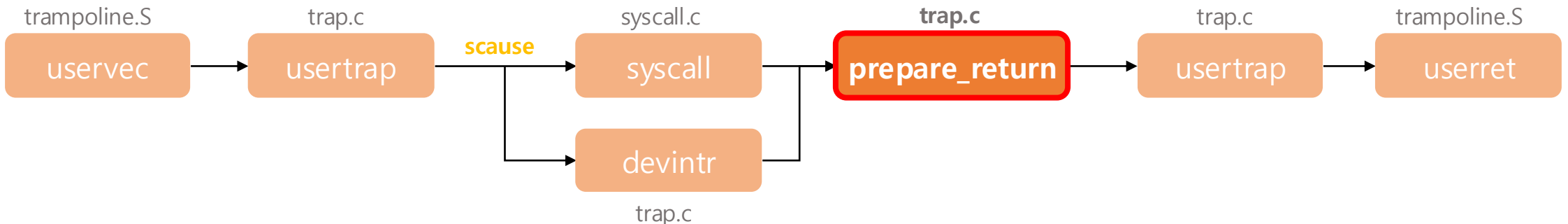


prepare_return

1. Disable interrupt.
2. Change **stvec** to point to **uservec**.
3. Save kernel execution data into **TRAPFRAME**.

```
$ vim kernel/trap.c

100 prepare_return(void)
...
107     intr_off();
108
109     // send syscalls, interrupts, and exce...
110     uint64 trampoline_uservec = TRAMPOLINE...
111     w_stvec(trampoline_uservec);
...
115     p->trapframe->kernel_satp = r_satp();
116     p->trapframe->kernel_sp = p->kstack + ...
117     p->trapframe->kernel_trap = (uint64)u ...
118     p->trapframe->kernel_hartid = r_tp();
```

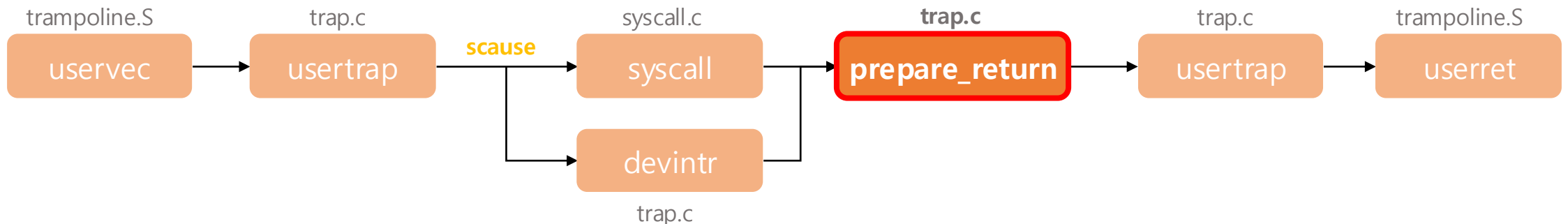


prepare_return

4. Restore **sstatus** to its previous state.
5. Set **sepc** to the saved user program counter.
6. Return to `usertrap()`.

```
$ vim kernel/trap.c

100 prepare_return(void)
...
123 // set S Previous Privilege mode to User.
124 unsigned long x = r_sstatus();
125 x &= ~SSTATUS_SPP; // clear SPP to 0 ...
126 x |= SSTATUS_SPIE; // enable interr ...
127 w_sstatus(x);
128
129 // set S Exception Program Counter to ...
130 w_sepc(p->trapframe->epc);
131 }
```

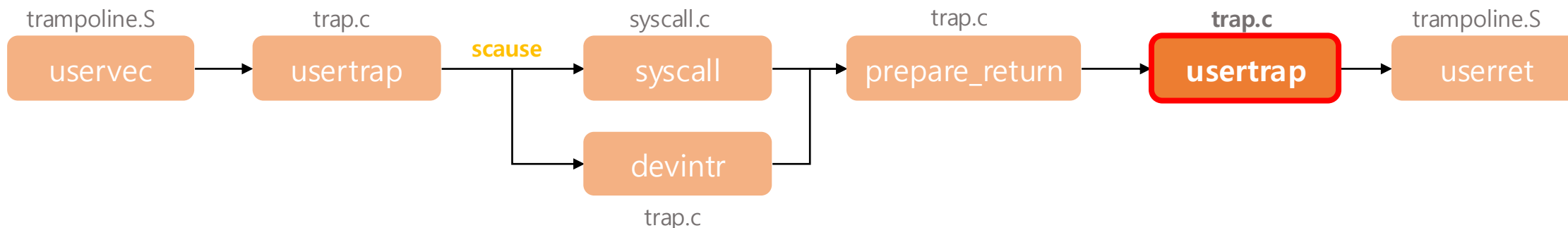


usertrap: after prepare_return

1. Return from `prepare_return()`.
2. Calculate **user page table** address.
3. Jump to `userret`.
 - passed via **a0 register**(=`return satp;`) to `userret`.

```
$ vim kernel/trap.c

38  usertrap(void)
...
87      prepare_return();
88
89      // the user page table to switch to, ...
90      uint64 satp = MAKE_SATP(p->pagetable);
91
92      // return to trampoline.S; satp value ...
93      return satp;
94 }
```



userret

1. Switch to the user page table.

- Receive page table address via **a0 register** from `usertrap()`.

```
$ vim kernel/trampoline.S
```

```
101 userret:
```

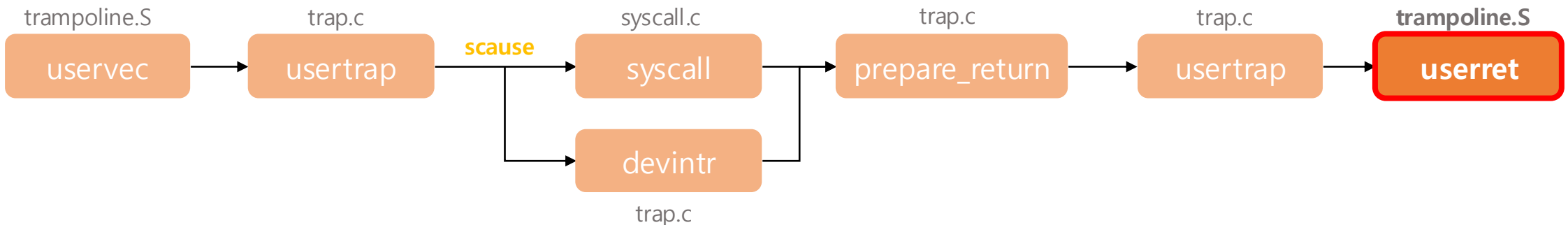
```
...
```

```
105 # switch to the user page table.
```

```
106 sfence.vma zero, zero
```

```
107 csrw satp, a0
```

```
108 sfence.vma zero, zero
```



userret

2. Restore registers from TRAPFRAME.
 - Restore a0 register at last.
3. Return to user mode and resume user program (restored from **sepc**).

```
$ vim kernel/trampoline.S
```

```
101 userret:
```

```
...
```

```
110 li a0, TRAPFRAME
```

```
...
```

```
113 ld ra, 40(a0)
```

```
114 ld sp, 48(a0)
```

```
...
```

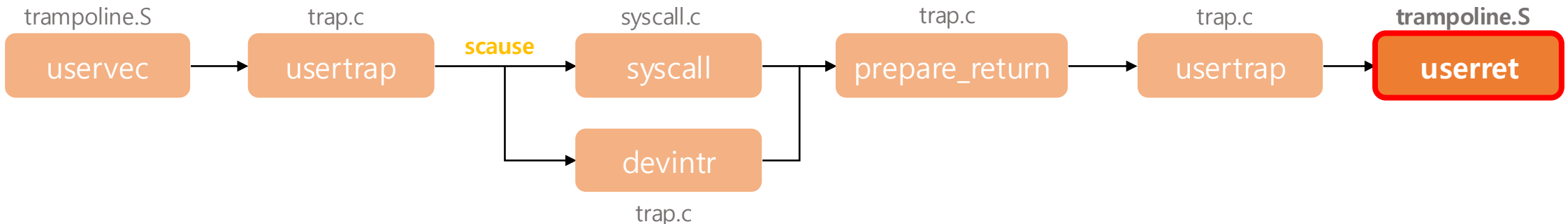
```
145 ld a0, 112(a0)
```

```
146
```

```
147 # return to user mode and user pc.
```

```
148 # usertrapret() set up sstatus and sepc.
```

```
149 sret
```



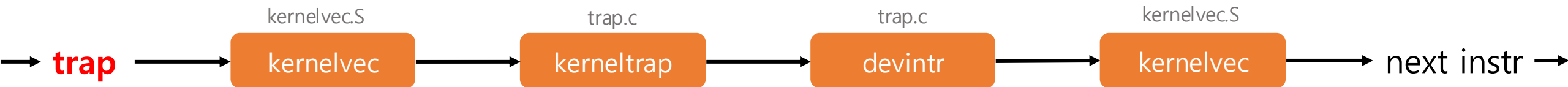
Traps from Kernel Space

kernel space trap flow

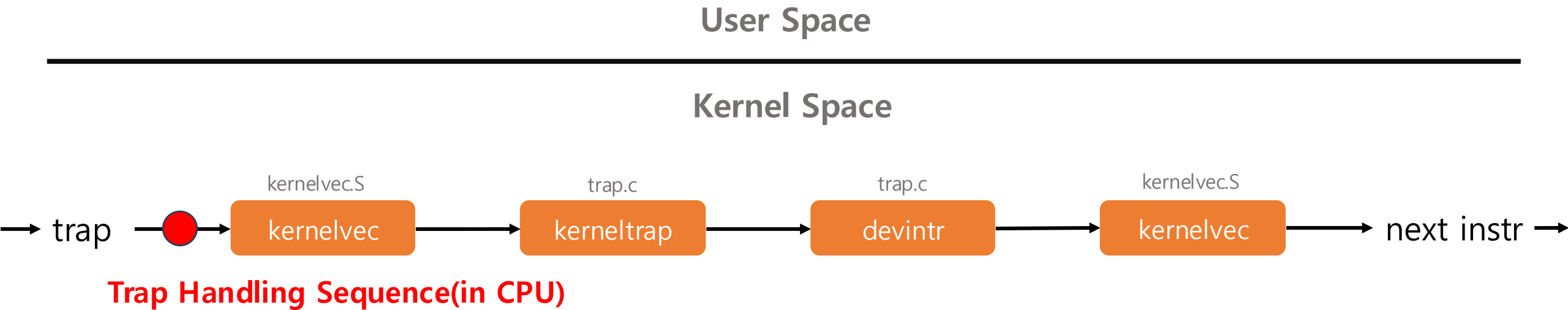
Traps from Kernel Space

User Space

Kernel Space



Traps from Kernel Space



Trap Handling Sequence

If the trap is a device interrupt, and the **sstatus** SIE bit is clear, don't do any of the following.

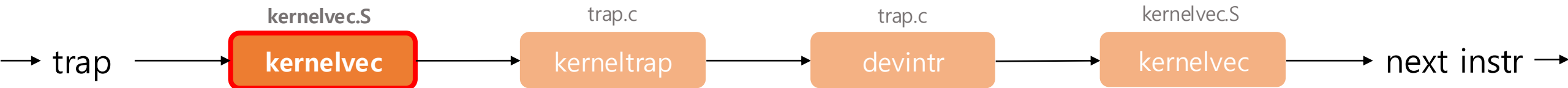
1. Disable interrupts by clearing the SIE bit in **sstatus**.
2. Copy the program counter to **sepc**.
3. Save the current mode (user or **supervisor**) in the SPP bit in **sstatus**.
4. Set **scause** to reflect the trap's cause.
5. Set the mode to supervisor.
6. Copy **stvec** to the program counter.
7. Start executing at the new program counter.

kernelvec

1. Save the kernel registers to the kernel stack.
2. Call `kerneltrap()`.

```
$ vim kernel/kernelvec.S
```

```
12 kernelvec:
13     # make room to save registers.
14     addi sp, sp, -256
15
16     # save the registers.
17     sd ra, 0(sp)
18     sd sp, 8(sp)
19     sd gp, 16(sp)
20     sd tp, 24(sp)
...
35     sd t6, 240(sp)
36
37     # call the C trap handler in trap.c
38     call kerneltrap
```

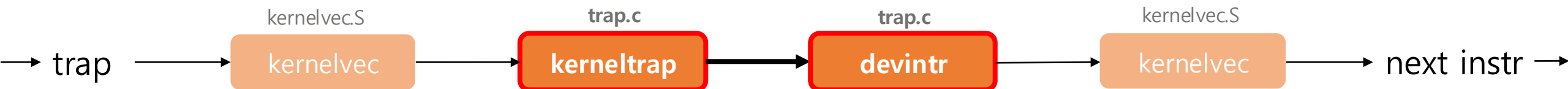


kerneltrap

1. Save values of **sepc**, **sstatus**, and **scause** registers to local variables.
2. Handle device interrupts with `devintr()`.
3. Restore the saved values back to the **sepc** and **sstatus** registers.

```
$ vim kernel/trap.c

139     uint64 sepc = r_sepc();
140     uint64 sstatus = r_sstatus();
141     uint64 scause = r_scause();
    ...
148     if((which_dev = devintr()) == 0){
149         // interrupt or trap from an ...
150         printf("scause=0x%lx sepc=0x%lx ...
    ...
154     // give up the CPU if this is a timer ...
155     if(which_dev == 2 && myproc() != 0)
156         yield();
    ...
160     w_sepc(sepc);
161     w_sstatus(sstatus);
```

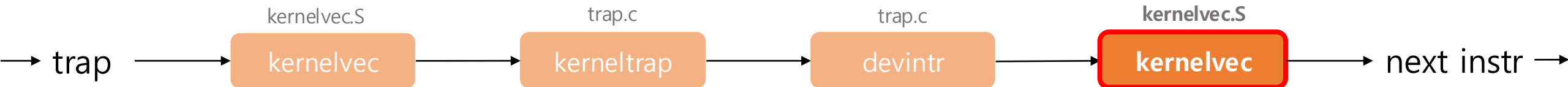


kernelvec: after kerneltrap

1. Return from `kerneltrap()`.
2. Restore the kernel registers from the kernel stack.
3. Resume the interrupted kernel execution.

```
$ vim kernel/kernelvec.S

12  kernelvec:
...
38      call kerneltrap
39
40      # restore registers.
41      ld ra, 0(sp)
42      ld sp, 8(sp)
...
59      ld t6, 240(sp)
60
61      addi sp, sp, 256
62
63      # return to whatever we were doing in ...
64      sret
```



Thank you

